

For this assignment, I used an adjacency list to represent the graph. Each node keeps a list of the nodes it's connected to. This works for a graph like this because it isn't fully connected everywhere so using an adjacency list saves a lot of space compared to something like an adjacency matrix, also it fits with BFS and DFS, Both algorithms constantly need to check a node's neighbors and an adjacency list makes that super quick, no scanning a giant matrix. So it's a simpler and faster way to store the graph for this kind of problem

Node 1	Node 2	BFS		DFS	
		Distance	Time (ms)	Distance	Time (ms)
N_0	N_1	2	0.3500	2	0.5844
N_0	N_2	3	0.0151	3	0.0301
N_0	N_3	5	0.0163	4	0.0299
N_0	N_4	24	0.0757	22	0.1652
N_0	N_5	6	0.0280	6	0.0550
N_0	N_6	4	0.0155	5	0.0303
N_0	N_7	7	0.0397	13	0.0871
N_0	N_8	16	0.0289	20	0.1471
N_0	N_9	21	0.0294	21	0.0911
N_0	N_10	8	0.0135	7	0.0258
N_0	N_11	11	0.0184	8	0.0302
N_0	N_12	9	0.0279	14	0.0541
N_0	N_13	12	0.0465	15	0.0584
N_0	N_14	17	0.0485	16	0.0802
N_0	N_15	10	0.0277	10	0.0375
N_0	N_16	15	0.0233	9	0.0508
N_0	N_17	13	0.0374	23	0.0711
N_0	N_18	18	0.0242	17	0.0583
N_0	N_19	22	0.0745	18	0.0694
N_0	N_20	14	0.0307	11	0.0625
N_0	N_21	20	0.0500	12	0.0429
N_0	N_22	19	0.0341	24	0.0885
N_0	N_23	23	0.0239	25	0.0795
N_0	N_24	25	0.0264	19	0.0275

1. Probably BFS, because BFS checks everything near the start first, so it finds nearby nodes almost instantly while DFS might wander way down some long path before ever coming back to the close node that someone actually wants. So for anything that's only a few hops away, BFS gets there faster and wastes less time, this matches what we see in the table, the BFS distances for close by nodes stay really small and consistent, while DFS sometimes overshoots cause it keeps exploring deeper branches before checking whats right next to the start.

2. DFS is probably the better pick because DFS goes straight down the path, which helps it reach deeper parts of the graph without having to look at a bunch of sideways branches first. BFS has to scan every level on the way down, which can add a lot of extra work. So if the target is buried deep in the graph, DFS can reach it with fewer steps and you can see this in the results. when the target node is far away DFS sometimes reaches it quicker even though BFS has to go layer by layer through a ton of nodes before it gets anywhere near that depth.